9.Structures

Se l'array permettono di definire le variabili con lo stesso tipo, le ***Structure*** è un altro metodo dove permette all'utente di definire delle variabili ma di DIVERSO TIPO (esempio)

```
struct Studente {
    char nome[50];  // stringa
    int eta;        // intero
    float media;    // float
};
```

Vengono usate per rappresentare dei record e per definire una structure , dobbiamo usare la dichiarazione dello struct

```
struct [tag] {
    member definition;
    member definition;
    member definition;
};
```

Il compilatore le distingue dalle variabili o dalle funzioni dove il quale il nome sia lo stesso della Structure tag, i suoi membri posso avere qualsiasi tipo desiderato , includendo anche dei tipi definiti in altre structure.
Spesso le structure possono contenere dei puntatori per il loro tipo.

Nell'ambito della definizione di un tipo di structure è possibile dichiarare oggetti di quel tipo in questo modo :

```
struct song song1,sogn2 *song = &song1;
```

è la parola chiave `struct` deve esser inclusa ogni volta che usiamo il tipo della structure , e possiamo anche usare il `typedef` per definire un nome per una structure:

```
typedef struct Song Song_t;
Song_t song1, song2, *pSong = &song1;
```

Possiamo a che definire dentro una structure un puntatore , nello stesso modo in cui definiamo un puntatore a una qualsiasi variabile .

accesso ai membri della structure

Ci sono due operatori che ci permettono di accedere ai membri di una structure :

- il punto `(.)`
- operatore freccia `–>)`

Copy structures

Possiamo anche usare l'assegnazione per copiare l'intero contenuto di un'oggetto structure in un altro stesso oggetto structure :

```
Books book1, book2; book2 = book1;
```

inizializzare una lista

per inizializzare una structure , dobbiamo semplicemente definire una lista : e questa è associata con l'ordinamento della dichiarazione dei membri e ogni inizializzatore deve avere un tipo che sia uguale al tipo del membro corrispondente

Unions

I membri di un union condividono tutti la stessa memoria di locazione , iniziano tutti con lo stesso indirizzo.
Le unions sono un semplice metodo per i programmatori per usare la locazione in memoria in un modo diverso.
La sua dichiarazione è simile a quella delle structure , eccetto la parola chiave union al posto di structure

```
union [tag] {
    member definition;
    member definition;
    member definition;
};
```

Un oggetto di questo tipo puo storare un intero , un numero floating point o un short string

```
union Data { int i; double x; char str[16]; };
```

Inizializzare unions

Come le structure , l'oggetti union sono inizializzati da un inizializzazione di una lista, e questa puo contenere un inizializzatore